

Cloud Computing - Exercise 03: Google Cloud Onboarding & Serverless App Development

Course: Cloud Computing **Exercise:** 03

Due Date: 07.12.2025, 23:55

1) Overview

In this third exercise, you will step into Google Cloud Platform (GCP) and build a small serverless application using Google-native services. You will:

- Set up a GCP student environment (using free credits or free trial) - with minimal or no out-of-pocket cost.
- Complete short, official Google Cloud learning modules to grasp GCP basics and serverless concepts.
- Develop a simple serverless **Cloud Function** that interacts with a **cloud database (Firestore)** to store and retrieve data, all on GCP.
- Deploy and test your function via an HTTP endpoint (no servers to manage), and view logs and data to verify it works.
- Practice cost control on GCP (stay within free tier limits or provided credits) and clean up all resources to avoid any charges.

Learning Objectives

By the end of this exercise, you should be able to: - **Navigate GCP and Set Up Services:** Create a GCP project and enable services (Cloud Functions, Firestore) using free student resources without incurring costs. - **Build a Serverless Function on GCP:** Develop and deploy a Google Cloud Function (in Python or Node.js) that runs on demand (HTTP trigger) and performs a simple task. - **Use a Managed Database (Firestore):** Initialize a Cloud Firestore database and use the function to write to and read from the database, demonstrating stateful data in a stateless app. - **Apply Security and IAM Best Practices:** Configure minimal required permissions for your cloud function via a service account (principle of least privilege) and allow safe public access to the function endpoint. - **Test and Monitor on GCP:** Invoke your function through its web URL (with query parameters for input), confirm it works (e.g. see expected output and stored data), and examine Cloud Logging for function execution details. - **Enforce Cost Awareness and Cleanup:** Utilize GCP's free tier or credits so no charges occur, set up any needed budget alerts, and properly delete or shut down all GCP resources after completing the exercise.

2) Prerequisites & Policies

- **Prior Exercises:** Ensure you have completed Exercises 01 and 02. You should be familiar with basic cloud concepts from AWS (S3, IAM, Lambda, etc.), which will help you in understanding analogous concepts in GCP.
- **Individual Work:** This assignment is to be done individually. You may discuss general approaches with classmates, but **do not share** your specific solution code, configurations, or screenshots. Every student must build and demo their own GCP setup and function.
- **Allowed Resources:** Official Google Cloud documentation, the course materials, and our course forum/office hours. If you use any external sources or references, cite them appropriately (except

provided course links). Avoid copy-pasting code from online sources - the function here is simple and provided in the instructions.

- **Zero-Cost Policy: €0 budget** - use only free resources. You must use GCP's Free Tier offerings or any educator-provided credits. **Do not enable** any paid services beyond the free allocations. Staying within the free usage limits is required. We will verify that your project incurs no cost.
- **Account Requirements:** A Google account is needed. GCP's free trial typically requires a credit/debit card *for verification only* (Google will not charge it unless you manually upgrade your account). If you have a promotional **education credit code** or access via the Google for Education program, you can redeem that to get free credits without a credit card (check with your instructor if such credits have been provided). **Do not proceed** with any setup that would bill you - ask for help if unsure.
- **Region Choice:** To maximize free-tier benefits and avoid latency issues, use a US region for this exercise (for example, **us-central1 (Iowa)**). GCP's always-free quotas for functions and Firestore apply per billing account (often globally or in US regions). Using us-central1 is recommended for consistency (it's broadly supported for many services and free-tier usage).
- **Service Limits:** This exercise stays within services readily available to new GCP accounts (Cloud Functions, Firestore, Cloud Logging). You should not need to request any quota increases. If a service is unavailable or you hit an unexpected limit, post on the forum.
- **Data Privacy:** Do not store any sensitive or personal data in GCP. The Firestore data you will create is just a simple test record (e.g. a greeting message), suitable for demonstration purposes only.
- **Ethical Use:** As with AWS, use GCP services ethically. Do not run any prohibited activities (e.g. crypto-mining, attacks, or scraping) in your cloud function or any part of your GCP project. Follow GCP's acceptable use policies at all times.

3) Set Up Your Google Cloud Environment (Free Account)

Goal: Gain access to Google Cloud Platform without incurring charges, using student-friendly options.

Use the GCP Free Trial (\$300 Credit)

If you do not have education credits, you can use GCP's standard free trial: 1. Go to GCP Free Trial - Start Free and sign in with your Google account. 2. Follow the sign-up steps. You will need to provide a credit or debit card **for identity verification**. **Note:** Google will grant you approximately **\$300 in trial credits for 90 days**. **You will NOT be charged** as long as you don't manually upgrade to a paid account. Ensure you **do not upgrade** your account or use services beyond the free credits. 3. Agree to the terms and activate the free trial. Once your account is set up, you should see your trial credit balance in the console. 4. Create a new project for this course work (if one isn't created by default). Name it something like "cloud-computing-ex3-<yourname>". Ensure this project is linked to the free trial billing account (check in **Billing** section of the console).

Post-Setup Checklist: - You have a GCP project with an active billing account (either the education credits account or the free trial account with credit). Remember the **Project ID** (format: organic-wharf-123456 or similar) as you'll need it for various steps. - Set your project in the console header to this new project. All actions must be in this project. - In the top-right of the console, confirm the project's billing is enabled and showing your credits. Also, confirm the project's region (for services that require region selection, we will specify us-central1). - If using the free trial, you may set up a budget alert now: go to **Billing -> Budgets & alerts**, create a budget for say €5 and an email alert (to ensure you are notified if any unexpected usage occurs). This is a good practice but not required if you stay within free allowances.

Troubleshooting: - *Credit redemption issues:* If your student credit link isn't working or you didn't receive one, contact the instructor immediately. Ensure you used your university email if required. - *Free trial sign-up issues:* If your card is declined or you can't sign up, you might try a different card or check if Google accepts other payment methods in your country (some regions allow PayPal). Google **will not deduct** money for the free trial itself. - *No access to console:* If you can't access the GCP Console due to permission or account issues, double-check you are logged in with the Google account you used for signup. In a pinch, you can use Google Cloud Shell (a web-based terminal) by visiting shell.cloud.google.com - it will activate for your project if you have an account.

4) Research-backed Micro-Learning

Before diving into building on GCP, complete the following two learning modules to familiarize yourself with the Google Cloud environment and serverless services. These are hands-on labs provided by Google, accessible via the **Google Cloud Skills Boost** platform. They will give you guided experience with the exact tools you'll use in the exercise.

A) Google Cloud Skills Boost - Cloud Functions: Qwik Start - Console

Why: This short lab introduces you to deploying a simple Cloud Function using the GCP Console. It covers the basics of writing a "Hello World" function and is directly relevant to our serverless function task.

How: Access the lab on the Skills Boost platform. You may find it by searching the catalog for "Cloud Functions: Qwik Start - Console" (Lab ID GSP081). Ensure you are logged in with your Skills Boost account - you have free credits as a student (either 200 Qwiklabs credits via Google's student offer or using the free tier labs). Start the lab, follow the instructions to deploy a sample function, and run it. The lab provides a temporary GCP environment, so **do not use your personal project for the lab** (the lab will provide its own project). The lab should take ~15 minutes.

Evidence: Upon completion, you will see a confirmation that you've finished the lab (usually a green check or score in Skills Boost). Take a **screenshot of the lab completion page** or your Skills Boost profile showing this lab as completed. The screenshot **must show** the lab name and that you completed it (and ideally your name or login in the corner of the screen for identification).

B) Google Cloud Skills Boost - Getting Started with Firebase Cloud Firestore

Why: This lab introduces Cloud Firestore (a NoSQL database in GCP) and how to perform basic read/write operations. It will help you understand how data can be stored and retrieved in a serverless app, which you'll implement in your exercise.

How: Find the lab titled "Getting Started with Firebase Cloud Firestore" (Skills Boost ID GSP1136). In this lab, you will use a provided sample web application to interact with Firestore. Follow the lab steps to initialize Firestore in a project, write a sample document, and then read it back using the web app. This will demonstrate how Firestore works in a simple scenario. The lab uses the Firebase and Firestore console and a bit of JavaScript coding in Cloud Shell. Expected duration is ~30 minutes. Make sure to use the provided lab environment (do not risk using your own project while in the guided lab).

Evidence: When finished, the lab will show a completion status. Take a **screenshot of the completion** (the final step or the score screen indicating success). Ensure the screenshot clearly shows the lab name "Firebase Cloud Firestore" and some indicator that you completed it (for example, a 100/100 score or a checkmark and your user name).

Complete both A and B. These modules are free for students and provide essential context. **Note:** If the Skills Boost platform asks for payment to start a lab, you likely have run out of credits or not applied the

student credits properly - seek help. Normally, the labs above are listed as free or should be covered by the credits given.

5) Hands-On Exercise - Serverless Greeting Service on GCP

Scenario: Now you will create a simple "**greeting service**" on Google Cloud. Similar to the AWS Lambda exercise (Exercise 02), this service will respond to HTTP requests with a greeting message. However, on GCP we'll add a twist: the service will also **remember** how many times it has greeted each name, by storing data in a database. Each time you greet the same name, it will increment a counter. You'll accomplish this using **Cloud Functions** for the serverless compute and **Cloud Firestore** for storage.

All tasks below assume you are working in your GCP project created in step 3, with billing enabled via free credits/trial. **Make sure you select the correct project** in the console before proceeding. Also set the region to **us-central1** for all services, unless instructed otherwise.

Step 1 - Enable APIs and Create a Firestore Database

GCP services often need their APIs enabled on your project before use.

- **Enable Cloud Functions API:** In the GCP Console, navigate to **APIs & Services → Library**. Search for "Cloud Functions API". Click on it and **enable** it for your project (if it isn't already enabled by default).
- **Enable Firestore API:** Still in the API Library, search for "Cloud Firestore API" (or "Datastore API", as Firestore is part of Datastore family). Enable this API as well.
- **Create Firestore Database:** Now, go to **Firestore** in the console (you can find it under Products → Databases → Firestore, or use the search bar to type "Firestore"). It will prompt you to select a mode and location:
 - Choose **Native mode** (not Datastore mode) for Firestore.
 - For region, select **nam5 (multi-region in US)** or **us-central** (if it allows a regional selection). The exact choices might vary; the key is to pick a US location to stay in free tier. If nam5 is available, use it (it's a multi-region US setting for Firestore).
 - Confirm creation. It may take a minute to set up the Firestore database.
 - Once Firestore is set up, you have a database ready. You don't need to create any collections or documents manually now - our Cloud Function will do that.

Checkpoint: In the Firestore console, you should see an empty database (no collections yet). Keep the console open or know how to return here to inspect data after running your function.

Step 2 - Create a Service Account with Minimum Permissions (IAM for Least Privilege)

By default, Cloud Functions in GCP run using your project's **default service account**, which often has broad editor permissions. To follow best practices (least privilege), you will create a dedicated service account for your function with narrowly-scoped permissions.

- Go to **IAM & Admin → Service Accounts** in the console.
- Click **" + CREATE SERVICE ACCOUNT "**. Provide a name like "cloud-function-sa" (the ID will auto-populate, e.g., cloud-function-sa).
- (Optional) Description can be "Service account for Exercise 03 Cloud Function".
- Click **Create and Continue**.

- In the next step, **Grant this service account access to project**:
- For **Select a role**, choose **Cloud Datastore -> Cloud Datastore User**. This role (roles/datastore.user) grants read/write access to Firestore (Firestore uses Datastore roles for IAM).
- Click **+ Add Another Role** and also add **Logs Writer** (under Cloud Logging). This ensures the function can write log entries. (If Logs Writer is not available in the list, you can find it under **Basic -> Project -> Editor** as a broad role, but *do not* use Editor. Instead, type "Logs Writer" in the role filter to find the specific role.)
- Ensure **Cloud Datastore User** and **Logs Writer** are listed as roles.
- Click **Done** to finish creating the service account. You should now see your new service account in the list (something like <project-number>-compute@developer.gserviceaccount.com if it's based on the default compute SA, or a custom one with your name if created anew).
- Copy or note the **email** of this service account (it will be something like cloud-function-sa@<your-project>.iam.gserviceaccount.com).

Checkpoint: In IAM, verify that the new service account has exactly the roles you gave it (you can click the service account, then Permissions). No need to add any more roles - we want to limit it strictly to Firestore use and logging.

Step 3 - Develop Your Cloud Function Code

We will write a Cloud Function that handles HTTP requests. If a request is made with a query parameter name, the function will greet that name and record the visit in Firestore. The Firestore will keep a count of how many times each name has been greeted.

We provide example code in **Python** (you may use Node.js if you prefer - see note below). The Python code will use the Google Cloud Firestore library.

Python Cloud Function code (example):

Create a file **main.py** with the following content:

```
from google.cloud import firestore

# Initialize Firestore client (uses application default credentials)
db = firestore.Client()

def hello_gcp(request):
    """HTTP Cloud Function that greets a name and tracks the count in Firestore."""
    # Parse the request
    request_args = request.args if request.args else {}
    name = request_args.get('name', 'World')

    # Reference a document in "greetings" collection with key = name
    doc_ref = db.collection('greetings').document(name)
    doc = doc_ref.get()
    count = 0
    if doc.exists:
        # If document exists, get current count
        data = doc.to_dict()
```

```

if data and 'count' in data:
    count = data['count']
    count += 1 # increment count for this name

# Save the updated count back to Firestore
doc_ref.set({'count': count})

# Log an info message (will appear in Cloud Logging)
print(f"Greeted {name} {count} time(s).")

# Return an HTTP response
return f"Hello, {name}! You have been greeted {count} time(s)."
```

And create a **requirements.txt** file with the following line (so that the Firestore library is available in the function environment):

```
google-cloud-firestore
```

Notes: - This function, `hello_gcp`, will be the entry point for Cloud Functions. It expects an HTTP request and returns a text response. - It retrieves a Firestore document under collection "greetings" named after the person (name). If none exists, `doc.exists` will be `False` and count remains 0. It then sets the new count (0+1 for first time, or current count+1 for subsequent). - It prints a log line with the count, which will go to Cloud Logging. - Finally, it returns a simple string. (Cloud Functions will interpret a returned Python string as an HTTP 200 response with that string as the body). - We use `firestore.Client()` with no arguments - this uses the default credentials (the service account attached to the function) to connect to the Firestore in our project.

If you prefer to use **Node.js** for the function, you can. Ensure you use equivalent logic (you'd use the Node Firebase Admin SDK or Google Cloud client to access Firestore). However, the provided Python code is recommended for simplicity, and we will assume Python in subsequent instructions.

Step 4 - Deploy the Cloud Function on GCP

Now deploy your function using the GCP Console:

- Navigate to **Cloud Functions** in the console (ensure your project is selected). Click "**CREATE FUNCTION**".
- **Basics:**
- **Function Name:** `helloGCP` (or any name you like, e.g., `greetingService`).
- **Region:** select **us-central1**.
- **Environment:** choose **1st gen** or **2nd gen** - either works for this simple use case. If unsure, select 1st gen for simplicity.
- **Trigger:** select **HTTP**. Check the box "**Require authentication**" **OFF** (in 1st gen, uncheck "Enable authentication"). In 2nd gen, it might say "Allow unauthenticated invocation" - make sure that is **allowed**. This is crucial so your function can be invoked by anyone (just like our AWS Lambda via API Gateway was public).
- **Runtime, Build and Connections:**
- **Runtime:** Choose **Python 3.10** (or latest 3.x available). If using Node.js instead, choose Node 18+.

- For **Entry point** (the function name in code): put `hello_gcp` (or the name of your function if you changed it).
- **Service Account:** By default it shows "Default". Click the dropdown and select the service account you created in Step 2 (`cloud-function-sa@...`). This ensures your function runs with limited permissions.
- You can leave other advanced settings as default (memory 256MB, etc.).
- **Code:**
- Select **Inline Editor** (should be default).
- In the file list, click to open **main.py** (it might already be open by default). Replace any sample code with the Python code from above.
- Find the **requirements.txt** in the file list. Open it and add the `google-cloud-firestore` line (if the file isn't present, create a new file named `requirements.txt` and add the line).
- Double-check: `main.py` has `def hello_gcp(request): ...` and `requirements.txt` has the dependency.
- **Deploy:**
- Click **Deploy** and wait. Deployment might take a couple of minutes (it has to install the Firestore library and set up the function).

If deployment succeeds, you'll see your function listed in Cloud Functions with a green check. If there are any errors (red X), click the function name and check the **Logs** or **Error** tab for details (common issues could be a typo, or forgetting to allow unauthenticated access - which you can fix by editing the function's permissions).

Checkpoint: You have a deployed function. In the function's detail page, note the **Trigger URL** - this is the HTTPS endpoint to call your function. It should look like `https://<region>-<project>.cloudfunctions.net/helloGCP` (for 1st gen) or a longer URL for 2nd gen.

Step 5 - Test the Function and Firestore Integration

Now it's time to invoke your function and see if it interacts with Firestore as expected.

- **Test via Browser:** Open a web browser in incognito (to avoid any cached credentials) and visit the function's URL. Append `?name=Alice` (or any name of your choice) as a query parameter. For example:
[https://us-central1-*<your-project>*.cloudfunctions.net/helloGCP?name=Alice](https://us-central1-<i><your-project></i>.cloudfunctions.net/helloGCP?name=Alice)
- When you hit enter, the function should execute and you should receive a response in the browser. It should be a plain text like:
"Hello, Alice! You have been greeted 1 time(s)."
- Refresh the URL (or open it again). The count should increment. The second time, you should see:
"Hello, Alice! You have been greeted 2 time(s)." Each refresh triggers the function again (HTTP GET requests), and it updates Firestore.
- Try another name: change the URL to `...?name=Bob`. The response should say Bob has been greeted 1 time (since Bob's count starts at 0).
- **Use the GCP Console to verify data:** Go to the Firestore section of the console. You should now see a collection **"greetings"** (created by the function on first run). Inside it, there will be documents for each name you greeted (e.g., "Alice", "Bob"). Click on one document (Alice) - you should see a field `count` with the latest number (e.g., 2). This confirms that the function successfully wrote to the database.

- Optionally, you can test the function using the built-in **Testing** tool in Cloud Functions console: select your function, go to the Testing tab. For an HTTP function, you can only provide the query parameters or body. Since our function reads request.args, simply provide a JSON with "query": e.g. { "query": "name=Charlie" } or use cURL. However, since it's easy via browser, that suffices.

Checkpoint: Take note of successful outcomes: - You have seen correct greetings and counts returned in the browser for different names. - The Firestore database has recorded the counts (persisting between calls). - The function endpoint is publicly accessible (no auth required due to our setting).

If the function call in browser resulted in an error or blank: - Make sure the function's permissions allow unauthenticated (in Cloud Functions details, there's a Permissions tab - it should list "allUsers" as a member with the Cloud Functions Invoker role). If not, add that permission. - Check the Cloud Function logs for errors (see next step).

Step 6 - Monitor Logs and Verify IAM Settings

Just like we used CloudWatch Logs for AWS, on GCP all function print statements and errors go to **Cloud Logging**.

- Go to **Logging → Logs Explorer** in the console. In the **Query** builder, set the resource to **Cloud Function** and choose your function name, and the severity to INFO. Alternatively, simply search for the text "Greeted" (since our code prints "Greeted X ...").
- You should find log entries for each invocation. For example, an entry might read: Greeted Alice 2 time(s). with a timestamp. This corresponds to our print statement in code.
- Also notice log entries labeled "Function execution started" and "Function execution finished" for each call - these are system logs indicating each run.
- Take a **screenshot of the Logs Explorer** showing at least one of your function's log entries (especially the custom print line with the count). Make sure the screenshot shows the log details and timestamp.
- (Optional) Check your function's IAM role: In Cloud Functions, go to your function's details → Configuration. It should show the service account it's running as (the one you created). If you click that service account in IAM, you'll see it has the roles we set (Datastore User, Logs Writer). This confirms least privilege: for instance, it does **not** have Editor or Owner rights on the whole project, meaning if your code tried to do something outside Firestore or logging, it might be blocked. Our function only needed those two capabilities and it works - this is good security practice.

Checkpoint: You have validated that logs are being recorded and that your IAM setup is functioning as intended (the function had no permission issues writing to Firestore or logs). If there were permission problems, you would see errors in the logs indicating access denied - in which case, recheck that you assigned the roles to the service account and that the function is using that service account.

Step 7 - Cleanup Steps

After capturing all required evidence, **delete your resources** to avoid any accidental costs:

- **Delete the Cloud Function:** In Cloud Functions console, click the menu for your function (three dots on the right) and select **Delete**. Confirm deletion. This will stop any further charges for Cloud Function invocations. (Cloud Functions incur a very tiny cost per invocation beyond free tier, but we stayed within free limits. Still, it's good practice to remove it when done.)

- **Delete Firestore Data:** Go to Firestore, expand the "greetings" collection. For each document (e.g., Alice, Bob), click the three dots and **Delete document**. Then delete the collection if it becomes empty (Firestore might not allow deleting a non-empty collection in one go, so remove documents first). *Note:* Firestore always has the database instance active (which doesn't cost anything just to exist). You cannot "delete the database" easily, but having an empty Firestore will ensure no ongoing usage beyond the free tier. It's fine to leave the Firestore provisioning itself as is, since empty DB won't incur charges.
- **Remove Service Account (optional):** If you want to tidy up, you can delete the custom service account since it's no longer needed. Go to IAM → Service Accounts, select cloud-function-sa and delete it. (Note: make sure no other resource is using it; in our case we deleted the function that was using it.)
- **Verify No Billing:** In your GCP console, check **Billing -> Reports** or the free tier usage. You should see \$0 charges. If you used the free trial, your \$300 credit should remain largely untouched (maybe a few cents of it "spent" but then covered by always-free tier, typically it'll still show full \$300 remaining). If you used an education credit, it might show a few cents of credit used - that's fine. The key is no actual charges beyond credits.
- (Optional) **Budget Alert or Project Deletion:** If you plan not to use this GCP project anymore, you can **disable billing** on the project or even delete the project entirely (Billing → disconnect account, and IAM → Settings → Shut down project). Shutting down the project will delete all resources (including any straggling Firestore info). Only do this if you're sure you won't use the project for anything else in the course.

By cleaning up, you ensure that you won't get unexpected bills and that you leave no unnecessary cloud resources running.

6) Submission Checklist

Include the following items in your submission on the LMS:

- **GCP Project ID and Region:** Clearly state the GCP project ID you used, and the region (should be us-central1) where your function was deployed.
- **Cloud Function Code:** Provide the name of your Cloud Function and a snippet or screenshot of your function code (main part). *(Do not just copy code text into your report; a screenshot is fine to show you implemented it. Ensure the code screenshot includes the key logic and maybe the requirements.txt).*
- **Function URL & Execution Evidence:** The URL of your Cloud Function's HTTP trigger. Also include a screenshot of your function's **response in the browser** for a sample query. For example, a screenshot of the page Hello, Alice! You have been greeted X time(s). as seen in your browser for one of the calls. (You can blur out any portion of your project ID in the URL if you want, but it's not necessary.)
- **Firestore Data Screenshot:** A screenshot of your Firestore console showing the **greetings collection** with at least one document and the count field. This is proof that the data was stored persistently. Make sure the document name (e.g., "Alice") and the count value are visible.
- **Logs Screenshot:** A screenshot from Cloud Logging showing your function's log entries (particularly the custom log "Greeted name X time(s)" line for one invocation, with timestamp). Ensure the function name and maybe a snippet of the log context (start/finish logs) are visible around it.
- **Training Modules Evidence:** Screenshot(s) for:

- *Cloud Functions: Qwik Start - Console* lab completion.
- *Firebase Cloud Firestore* lab completion. Each screenshot must clearly show the module name and an indication of completion (and your account name or email on the platform).
- **Reflection:** A short text (100-200 words) reflecting on this exercise. Describe **what you learned** by using GCP (differences or similarities to AWS, new concepts like service accounts or Firestore, etc.), mention **one challenge or pitfall** you encountered (for example, any setup issues or debugging hurdles), and **one question or point of confusion** that remains (perhaps about GCP services or something you'd like to explore more).

Make sure all screenshots are legible. You can annotate them if needed with highlights or arrows. Double-check that you haven't captured any sensitive info in screenshots (though generally project IDs and function URLs are okay to share for this assignment context).

7) Grading Rubric (100 points)

- **20 pts - Environment Setup & Governance:** GCP account is correctly set up (using free credits/trial), and project configured in recommended region. Evidence of cost-awareness (e.g., budget alerts or notes on free tier) and proper cleanup is provided. No unnecessary resources left running.
- **20 pts - Learning Module Completion:** Valid screenshots provided for both required Skills Boost modules (Cloud Functions lab and Firestore lab). Screenshots clearly show the student's identity and completion status.
- **30 pts - Cloud Function Implementation and Functionality:** The Cloud Function is deployed and works as specified:
 - Correct logic to handle name queries and integrate with Firestore (increment counts).
 - HTTP trigger publicly accessible and returns expected greeting message. Firestore data updates correctly with each call.
 - Evidence (function output screenshot and Firestore data screenshot) supports the correct operation.
- **10 pts - Security and IAM Configuration:** The function is run with a custom service account with limited permissions (Firestore user role). Unauthenticated access is enabled for the function (so it's callable publicly). The student followed least privilege practices (as seen in their description or IAM evidence). Full credit even if default SA was used *only* if everything else is correct and no security risk in this context, but the intent is to see an attempt at least privilege.
- **10 pts - Logging and Monitoring:** Cloud Logging screenshot shows the function's execution logs, indicating the student verified the function's runtime behavior. The logs should include the custom message or other evidence of successful invocation. Student demonstrates understanding of how to find and use logs.
- **10 pts - Reflection Quality:** The reflection is thoughtful and within 100-200 words. It clearly states new learnings (especially any insight about using GCP vs AWS), identifies a challenge faced during the exercise, and poses an open question or area of uncertainty. Writing should be clear and concise.

Deductions may occur for missing or wrong evidence, not following instructions (e.g., if resources were left running or if the function doesn't actually use Firestore when it should), or if the submission lacks clarity/organization.

8) Safety, Cost & Cleanup

This exercise is designed to be free-tier friendly on GCP:

- **Cloud Functions Free Tier:** GCP offers **2 million free invocations per month** and a generous amount of CPU time and networking for Cloud Functions. Our use (a handful of function calls, a few milliseconds each) is negligible compared to these limits.
- **Firestore Free Tier:** Firestore provides **1 GB of storage and 20,000 writes/50,000 reads per day for free**. Our usage (a few documents and increments) is well within the free daily quota. The small number of documents (and their tiny size) will be far under 1 GB.
- **Always-Free Services:** Other services we used, like Cloud Logging, Cloud Shell, etc., have free usage for basic operations. For instance, Cloud Logging's ingestion of small volumes is free up to certain limits that we won't exceed in this exercise.
- **Credits:** If you used the \$300 free trial, the credit covers all usage. If you used a \$50 student credit, our exercise might use only a few cents of it (if at all, given the free tier covers most of it). In either case, you should see **no actual charges** to any card.

Despite the above, it's important to be vigilant: - We recommended setting up a budget alert for an extra safety net. - If at any point you see any charge in the billing section, report it immediately and explain in your submission if it happened (it shouldn't if instructions were followed). - Always **shut down** resources when done. In cloud, forgetting a running service (like a VM or a high-throughput function) can lead to costs. In our case, deleting the function and clearing Firestore ensure nothing will incur costs later. - By cleaning up, you also practice good cloud hygiene - an important skill to avoid clutter and surprises in real projects.